

Week 2: Context Free Grammars

** 1. Design CFGs for the following programming language lexemes over the ASCII alphabet. You will find it convenient to use abbreviations like $\dots$ to help present the rules compactly.

- (a) A *C program identifier* is any string of length at least 1 containing only letters ('a'-'z', lower and uppercase), digits ('0'-'9') and the underscore, and which begins with a letter or the underscore.
- (b) An *integer literal* is any string taking one of the following forms:
 - a non-empty sequence of digits (decimal)
 - a non-empty sequence of characters from '0'-'9', 'a'-'e' (upper or lowercase) that are preceded by "0x" (hexadecimal)
 - a non-empty sequence of bits '0' and '1' that are preceded by "0b" (binary)

Solution

- (a) This is one way to describe it, I will write terminal symbols in terminal type to distinguish them:

$$\begin{aligned} S &::= LM \\ M &::= LM \mid DM \mid \epsilon \\ L &::= a \mid b \mid \dots \mid z \mid _ \mid A \mid B \mid \dots \mid Z \\ D &::= 0 \mid 1 \mid \dots \mid 9 \end{aligned}$$

- (b) One straightforward way is as follows (here I use terminal type for the terminal symbols to distinguish them from nonterminals):

$$\begin{aligned} S &::= D \mid H \mid B \\ D &::= ND \mid N \\ N &::= 0 \mid 1 \mid \dots \mid 9 \\ H &::= 0xG \\ G &::= AG \mid A \\ A &::= N \mid a \mid b \mid \dots \mid e \mid A \mid B \mid \dots \mid E \\ B &::= 0bZ \\ Z &::= YZ \mid Y \\ Y &::= 0 \mid 1 \end{aligned}$$

** 2. Give a CFG that describes the language of all subsequences of all permutations (i.e. no repetition)

of the following keywords:

final static synchronized

Solution

$$\begin{aligned} S &::= ABC \mid ACB \mid BAC \mid BCA \mid CAB \mid CBA \\ A &::= \text{final} \mid \epsilon \\ B &::= \text{static} \mid \epsilon \\ C &::= \text{synchronized} \mid \epsilon \end{aligned}$$

- * 3. Consider the following grammar, which describes the structure of statements in an imperative programming language.

$$\begin{aligned} \text{Prog} &::= \text{Stmt Stmt} & (1) \\ \text{Stmt} &::= \text{if exp then Stmt else Stmt} & (2) \\ & \mid \text{while exp do Stmt} & (3) \\ & \mid \text{skip} & (4) \\ & \mid \text{id} \leftarrow \text{exp} & (5) \\ & \mid \{ \text{Stmt Stmt} \} & (6) \\ \text{Stmts} &::= ; \text{Stmt Stmt} & (7) \\ & \mid \epsilon & (8) \end{aligned}$$

In it expressions appear only as a terminal symbols *exp* because the structure of expressions is not important to the exercises. In total, the terminal symbols are: if, then, else, while, do, skip, id, exp, the left and right braces, the end-of-input marker, the assignment backwards-arrow and the semicolon. The rules are numbered to make constructing the parse tables easier. The language of this grammar (start symbol *Prog*) includes strings such as:

while exp do id $\leftarrow$ exp

i.e. strings that show the control structure of the program without specifying the particular expressions involved.

The nullable, first and follow maps for the nonterminals in this grammar are as follows:

Nonterminal	Nullable?	First	Follow
Prog	×	if, while, skip, id, {	
Stmt	×	if, while, skip, id, {	else, ;, }
Stmts	✓	;	}

- For each rule $X ::= \alpha$ numbered (1) – (8), compute $\text{First}(\alpha)$, the set of terminal symbols that can start any string derivable from the rule right-hand side α .
- Construct the parsing table for the grammar.
- Is the grammar LL(1)?

Solution

(a)

$\text{First}(\text{Stmt Stmt}) = \text{if, while, skip, id}$
 $\text{First}(\text{if exp then Stmt else Stmt}) = \text{if}$
 $\text{First}(\text{while exp do Stmt}) = \text{while}$
 $\text{First}(\text{skip}) = \text{skip}$
 $\text{First}(\text{id} \leftarrow \text{exp}) = \text{id}$
 $\text{First}(\{\text{Stmt Stmt}\}) = \{$
 $\text{First}(\text{; Stmt Stmt}) = \text{;}$
 $\text{First}(\epsilon) =$

(b)

Nonterminal	if	exp	then	else	while	do	skip	id	;	{	}	←
Prog	1				1		1	1		1		
Stmt	2				3		4	5		6		
Stmts									7		8	

(c) Yes.

* 4. Consider now the following grammar:

$\text{Prog} ::= \text{Stmt Stmt} \quad (1)$
 $\text{Stmt} ::= \text{if bexp then Stmt else Stmt} \quad (2)$
 $\quad \quad \quad | \text{while bexp do Stmt} \quad (3)$
 $\quad \quad \quad | \text{skip} \quad (4)$
 $\quad \quad \quad | \text{id} \leftarrow \text{aexp} \quad (5)$
 $\quad \quad \quad | \text{Stmt Stmt} \quad (6)$
 $\text{Stmts} ::= \text{; Stmt Stmt} \quad (7)$
 $\quad \quad \quad | \epsilon \quad (8)$

This grammar is the same as the previous one, except that braces have been removed in rule 6. The Nullable, First and Follow maps for this grammar can be tabulated as follows.

Nonterminal	Nullable?	First	Follow
Prog	×	if, while, skip, id	
Stmt	×	if, while, skip, id	;, else
Stmts	✓	;	;, else

- For each rule $X ::= \alpha$ numbered (1) – (8), compute $\text{First}(\alpha)$, the set of terminal symbols that can start any string derivable from the rule right-hand side α .
- Construct the parsing table for this grammar.
- Is the grammar LL(1)?

(a)

- (1) $\text{First}(\text{Stmt Stmt}) = \text{if, while, skip, id}$
- (2) $\text{First}(\text{if exp then Stmt else Stmt}) = \text{if}$
- (3) $\text{First}(\text{while exp do Stmt}) = \text{while}$
- (4) $\text{First}(\text{skip}) = \text{skip}$
- (5) $\text{First}(\text{id} \leftarrow \text{exp}) = \text{id}$
- (6) $\text{First}(\text{Stmt Stmt}) = \text{if, while, skip, id}$
- (7) $\text{First}(\text{; Stmt Stmt}) = \text{;}$
- (8) $\text{First}(\epsilon) =$

(b) The parsing table is as follows:

Nonterminal	if	bexp	then	else	while	do	skip	id	aexp	;	$\leftarrow$
Prog	1				1		1	1			
Stmt	2, 6				3, 6		4, 6	5, 6			
Stmts				8						7, 8	

(c) No.

** 5. Consider the following grammar for arithmetic expressions:

- $$\begin{aligned}
 E &::= \text{num} & (1) \\
 &| \text{id} & (2) \\
 &| E + E & (3) \\
 &| E * E & (4) \\
 &| (E) & (5)
 \end{aligned}$$

The 6 terminal symbols of this grammar are:

num id + * ()

The nullable, first and follow sets for the nonterminals are:

Nonterminal	Nullable?	First	Follow
E	×	num, id, (	+, *,)

(a) For each of the rules (1) – (5), construct the first set of the right-hand side of the rule.

(b) Construct the parsing table for this grammar.

(c) What do the rule conflicts (cells of the parsing table containing more than one rule) of this grammar and the previous grammar have in common? Without constructing the parsing table, argue that the following grammar is sure to have a rule conflict:

- $$\begin{aligned}
 X &::= a \\
 &| b \\
 &| X c
 \end{aligned}$$

Solution

(a)

$\text{First}(\text{num}) = \text{num}$
 $\text{First}(\text{id}) = \text{id}$
 $\text{First}(E + E) = \text{num, id, (}$
 $\text{First}(E * E) = \text{num, id, (}$
 $\text{First}((E)) = ($

(b)

Nonterminal	num	id	+	*	(	)
E	1,3,4	2,3,4			3,4,5	

- (c) Whenever a non-terminal X appears as the leftmost symbol in its own production rule $X ::= X \beta$, and $\text{First}(X)$ is non-empty, then there is certain to be a conflict. Since $\text{First}(X)$ is not empty, there must be some terminal symbol $a \in \text{First}(X)$ and another rule $X ::= \gamma$ with $a \in \text{First}(\gamma)$ – intuitively, this rule is the reason that a is contained in $\text{First}(X)$. Then, since $\text{First}(X \beta) \supseteq \text{First}(X)$, it is certain that $\text{First}(X \beta)$ and $\text{First}(\gamma)$ will both contain a , and thus the cell of the parse table at coordinate (X, a) will contain both rules.

* 6. Consider the following grammar for arithmetic expressions, whose terminals are as above:

$E ::= \text{num } F$
 $\quad | \text{id } F$
 $\quad | (E) F$
 $F ::= + E$
 $\quad | * E$
 $\quad | \epsilon$

The nullable, first and follow sets can be described by:

Nonterminal	Nullable?	First	Follow
E	×	num, id, (	)
F	✓	+, *	)

- (a) Construct the first set of each of the right-hand sides of rules in the grammar.
- (b) Construct the parsing table of the grammar.
- (c) Is the grammar LL(1)?

Solution

(a)

First(num F) = num
First(id F) = id
First((E) F) = (
First(+ E) = +
First(* E) = *
First(ϵ) =

(b)

Nonterminal	num	id	+	*	(	)
E	1	2			3	
F			4	5		6

(c) Yes.

** 7. Give CFGs for the following languages. The later parts are more difficult than 2-star.

- (a) All odd length strings over $\{a, b\}$.
- (b) All strings over $\{a, b\}$ that contain aab as a substring.
- (c) The set of strings over $\{a, b\}$ with more a than b . Hint: every string w with at least as many a as b (possibly the same number of a as b) can be characterised inductively as follows. Either:
- w is just a
 - or, w is of shape avb with v containing at least as many a as b
 - or, w is of shape bva with v containing at least as many a as b
 - or, w is of shape v_1v_2 with v_1 and v_2 each separately containing at least as many a as b
 - or, w is the empty string
- (d) The complement of the language $\{a^n b^n \mid n \geq 0\}$ over $\{a, b\}$. Hint: express "not of shape $a^n b^n$ for some n " into one or more positive (i.e. without using *not* or similar) conditions.
- (e) $\{v\#w \mid v, w \in \{a, b\}^* \text{ and the reverse of } v \text{ is a substring of } w\}$, over $\{a, b, \#\}$.

Solution

(a)

$S ::= XSX \mid X$
 $X ::= a \mid b$

(b)

$S ::= XSX \mid aab$
 $X ::= a \mid b \mid \epsilon$

- (c) Here we dedicate a nonterminal T to describe the inductive characterisation given in the hint and then use S to force an extra a .

$$\begin{aligned} S &::= TaT \\ T &::= a \mid aTb \mid bTa \mid TT \mid \epsilon \end{aligned}$$

- (d) When you are asked to give the complement, or otherwise to describe something according to a negated condition, it is best to translate it immediately into one or more positive conditions because CFG have no direct way to express negation. In this case, not of shape $a^n b^n$ means either:

- The word has all the a before any b but there are strictly more a than b (nonterminal R),
- or, the word has all the a before any b , but there are more b than a (nonterminal T),
- or, the word has a b occurring before an a (non-terminal U).

$$\begin{aligned} S &::= R \mid T \mid U \\ R &::= AaRb \mid a \\ A &::= aA \mid \epsilon \\ T &::= aTbB \mid b \\ B &::= bB \mid \epsilon \\ U &::= XbXaX \\ X &::= aX \mid bX \mid \epsilon \end{aligned}$$

- (e) Strings in this language have the following shape: $v\#w_1v^Rw_2$ where w_1 and w_2 are completely arbitrary. We need to match v to v^R letter by letter, so this is best done by a single nonterminal (T). The “middle” of the string generated by this non-terminal (between the matching pieces of v and its reverse) will contain the $\#$ and, to the right of it, any arbitrary substring w_1 . Finally, the whole string has an arbitrary suffix w_2 . We can describe this as follows:

$$\begin{aligned} S &::= TX \\ X &::= aX \mid bX \mid \epsilon \\ T &::= aTa \mid bTb \mid \#X \end{aligned}$$